Rahul Dange

+61 412 954 045 · contact@rahuldange.com · rahuldange.com
linkedin.com/in/rahuldangeofficial · github.com/rahuldangeofficial
Sydney, Australia · Full work rights (Subclass 485 Temporary Graduate visa)

Summary

Software engineer focused on systems and backend development. Experienced building concurrent C11 software involving memory management, atomics, POSIX threads and low-level debugging. Master of Computer Science from the University of Sydney and current Systems Programming tutor. Seeking a graduate or junior backend, systems or platform engineering role.

Technical Skills

Programming: C, C++, Python, JavaScript, SQL, x86-64 Assembly
Systems: C11 atomics, POSIX threads, memory management, system calls, signals, IPC, Linux
Debugging & Testing: GDB, Valgrind, AddressSanitizer, ThreadSanitizer, UndefinedBehaviorSanitizer
Backend & Web: Node.js, Express.js, REST APIs, PostgreSQL, MongoDB, React.js
Tools: Git, Make, Bash

Experience

Academic Tutor – Systems Programming (COMP2017) Sydney, Australia
The University of Sydney *Feb 2026 – Present*

- Teach C systems programming to undergraduate students: memory layout, pointers, structures, stack and heap.
- Run tutorials on concurrency with POSIX threads, covering mutexes, semaphores, condition variables and race conditions.
- Help students debug segmentation faults and memory leaks using GDB and Valgrind.

Projects

MIMFS – Multithreaded In-Memory File System | **C11, POSIX Threads**

- Thread-safe POSIX-style file system that works inside a memory buffer the caller provides, with no dynamic allocation. Supports directories, hard and symbolic links, sparse files and rename.
- Block and inode allocation is lock-free using C11 atomics, with per-inode locks rather than a single global lock, to reduce contention between concurrent operations.
- Unit and concurrency test suites, run under AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer.

LLFPL – Low-Level Floating-Point Language | **C11**

- Interpreter for a deterministic expression language aimed at numerical work. Memory-maps the source and evaluates expressions directly without constructing a syntax tree, minimising intermediate memory use.
- Reserves all storage at start-up so evaluation performs no heap allocation, and keeps arithmetic to IEEE 754 double, so a program gives the same result on every run.
- Tested against expected-output files, including error cases, and run under sanitizers.

Education

The University of Sydney Sydney, Australia
Master of Computer Science (Software Engineering) *Feb 2024 – Dec 2025*

Nutan College of Engineering and Research (NCER) Pune, India
Bachelor of Technology in Computer Science and Engineering *Aug 2019 – Aug 2023*

Publication

Revolutionizing Student Engagement: Building a Cutting-Edge Interaction System. SSRN (Elsevier), 5th International Conference on Communication & Information Processing (ICCIP-2023).